Contents

MS Thesis Advisor Allocation — User Manual

---

1. Installation

1.1 Prerequisites

Two tools must be installed before you begin.

Tool	Mac	Linux	Windows
Git	`brew install git` or git-scm.com	`sudo apt install git` / `sudo dnf install git`	git-scm.com — install with default options
Conda	Miniconda (recommended)	Same	Same — use **Anaconda Prompt** for all commands below

> **Windows users:** run every command in this manual from **Anaconda Prompt**, not the default Command Prompt.

1.2 Step 1 — Clone the repository

Mac / Linux:

```
git clone https://github.com/aitgcodes/Allocator.git
cd Allocator
```

Windows:

```
git clone https://github.com/aitgcodes/Allocator.git
cd Allocator
```

1.3 Step 2 — Create the Conda environment

This installs Python 3.11 and all required packages into an isolated environment named `allocator`. It only needs to be done once.

```
conda env create -f environment.yml
```

> This may take a few minutes on the first run.

1.4 Step 3 — Activate the environment

Mac / Linux:

```
conda activate allocator
```

Windows:

```
conda activate allocator
```

Your prompt will change to show `(allocator)` on the left, confirming the environment is active.

1.5 Step 4 — Launch the app

Mac / Linux:

```
PYTHONPATH=src python -m allocator.app
```

Windows:

```
set PYTHONPATH=src && python -m allocator.app
```

You should see:

```
Dash is running on http://127.0.0.1:8050/
```

Open **http://localhost:8050** in Chrome, Firefox, or Edge. Keep the terminal open while using the app — closing it stops the server.

1.6 Stopping and restarting

Press **Ctrl+C** in the terminal to stop the app.

To restart later, the environment is already installed — only steps 3 and 4 are needed:

Mac / Linux:

```
cd Allocator
conda activate allocator
PYTHONPATH=src python -m allocator.app
```

Windows:

```
cd Allocator
conda activate allocator
set PYTHONPATH=src && python -m allocator.app
```

1.7 Troubleshooting installation

Symptom	Fix
`conda: command not found`	Miniconda/Anaconda is not installed or not on PATH — reinstall and restart your terminal
`ModuleNotFoundError: No module named 'allocator'`	`PYTHONPATH=src` was not set — re-run the launch command exactly as shown
Browser shows “This site can’t be reached”	The app is not running — check the terminal for errors and ensure Step 4 completed successfully
Port 8050 already in use	Another instance is running; stop it with `Ctrl+C`, or run `lsof -i :8050` (Mac/Linux) to find and kill it
`CondaEnvException: prefix already exists`	Environment already created — skip Step 2 and go straight to Step 3

---

2. Usage Guide

2.1 Preparing input files

The app requires two CSV files: one for students and one for faculty.

Option A — Convert a Google Form export Option A1 — In-app (recommended): Upload the raw form export directly to the app and click **Clean & Load**. The app normalises column names (`Roll No.` → `student_id`, `CPI (as on date)` → `cpi`, `Preference  N` → `pref_N`), maps faculty names to IDs, deduplicates repeated preferences, and backfills any missing faculty alphabetically — all in one step. A faculty CSV with IDs and names must still be uploaded alongside.

Option A2 — Offline script: Use the bundled script to produce both files before opening the app:

```
python scripts/make_preference_sheet.py form_responses.csv
```

This generates:

Output file	Contents
`preference_sheet.csv`	One row per student — `student_id`, `name`, `cpi`, `pref_1 … pref_N` (faculty IDs in preferences)
`faculty_list.csv`	One row per faculty — `faculty_id`, `name`, `max_load` (blank; fill in manually)

The script applies the same cleaning steps as Clean & Load: flexible column naming, deduplication, and alphabetical backfill.

Script options:

Flag	Default	Description
`-o / --output`	`preference_sheet.csv`	Student output path
`-f / --faculty-output`	`faculty_list.csv`	Faculty output path
`--cpi-col COLUMN`	auto-detect	Override CPI column name if auto-detection fails

Option B — Create files manually students.csv

Column	Description
`student_id`	Unique identifier
`name`	Full name
`cpi`	Cumulative Performance Index (numeric)
`pref_1`, `pref_2`, …	Faculty IDs in preference order

faculty.csv

Column	Description
`faculty_id`	Unique ID (must match values used in student preference columns)
`name`	Full name
`max_load`	*(optional)* Maximum students this advisor can take; leave blank to use the formula value `floor(S/F) + 1`

Sample files are provided in `data/` (`sample_students.csv`, `sample_faculty.csv`).

2.2 Running an allocation

Step 1 — Open the app Navigate to **http://localhost:8050**. The landing page shows a policy selector and two buttons:

- **Continue** → — proceed to the allocation workflow with the selected policy.
- **View Analysis** — jump directly to the Analysis page to load and compare previously saved runs, without starting a new allocation.

Step 2 — Select a policy Choose one of the five allocation policies on the landing page:

- **Least Loaded (`least_loaded`)** — the default; assigns each student to the least-loaded eligible advisor within their tier preference window.
- **Adaptive LL (`adaptive_ll`)** — like `least_loaded` but auto-tunes tier caps to guarantee no empty labs when $S \geq F$.
- **CPI Fill (`cpi_fill`)** — merit-first; processes students in strict descending CPI order; no tier window applied.
- **CPI-Tiered Preference Rounds (`tiered_rounds`)** — round-based; in round n each student offers their n-th preference; manual pick required at every round (auto-run also available).
- **Tiered LL (`tiered_ll`)** — runs tiered rounds 1..k (critical round k determined by dry-run), then switches to a two-phase backfill: Phase 2a assigns excess students (one per advisor, CPI order) while unassigned > empty labs; Phase 2b fills each remaining empty lab with the student's highest-preferred empty advisor. Guarantees no empty labs when $S \geq F$ and the preference structure is feasible.

A full discussion of all policies is in Section 3. To switch policy mid-session, click **Home** (top-right of the allocation page) to return to the landing page.

Step 3 — Load input files Use the file upload controls on the main page to upload your `students.csv` and `faculty.csv`. The app validates both files and reports any errors (missing columns, mismatched faculty IDs, etc.) before proceeding.

Two loading modes are available:

Button	When to use
Clean & Load	Raw Google Form export with faculty names in the preference columns — the app converts names to IDs automatically
Load directly	Pre-processed file with faculty IDs (`F01`, `F02`, …) already in the preference columns

Step 4 — Run Phase 0 Click **Run Phase 0 only** to tier students and compute faculty capacities without running the allocation, or click **Run full allocation** to do both in one step.

Step 5 — Complete the allocation

- **`least_loaded` / `adaptive_ll`:** Confirm Round-1 picks (default: highest-CPI student per advisor), then proceed to main allocation, making or confirming each assignment.
- **`cpi_fill`:** Run Phase 1 (manual or auto), then Phase 2.
- **`tiered_rounds`:** After Phase 0, choose manual or auto-run. In manual mode, each round presents per-advisor dropdowns for operator picks; confirm to advance. In auto-run mode, the engine resolves ties by CPI automatically.
- **`tiered_ll`:** After Phase 0, the dry-run computes the critical round k (the last round where running one more round would not leave more empty labs than remaining students). Run tiered rounds 1..k manually or automatically. On reaching round k the UI switches automatically to the two-phase backfill: Phase 2a (one student per advisor, CPI order, while unassigned > empty labs) followed by Phase 2b (each student to their highest-preferred empty lab).

2.3 The allocation replay panel

After the allocation runs, the replay panel at the bottom of the page shows five tabs, all linked to the **step slider**. Drag the slider or click **Play** to step through every assignment decision.

Tab	What it shows
Assignment Graph	Bipartite graph: students (left, sorted by CPI) connected to their assigned advisor (right). The most recently assigned edge is highlighted in purple.
Advisor Loads	Horizontal bar chart of current student count per advisor. Bars are coloured green / yellow / red by utilisation; the advisor just assigned is highlighted. A dashed red line marks `max_load`.
Statistics	Per-tier assignment counts and percentages, plus faculty load statistics (min, max, mean).
Step Log	Table of every allocation event up to the current step — step number, phase, and a one-line description of the assignment made.
CPI Distribution	Histogram of advisor-averaged CPIs (mean CPI of each advisor's assigned students). Advisors with no students yet appear in the 0 bin. The distribution shifts and spreads as the allocation progresses.

Tab	What it shows
Tier Heatmap	Advisor × CPI-tier heatmap. Each cell = students in that tier × 100 / capacity (`max_load`), so the row sum equals the advisor's total load as a percentage of capacity. Rows sorted by load descending. Diagnostic view for advisor equity — use together with Avg CPI Entropy.

2.4 Completion panel

When the allocation finalises, a completion panel appears above the replay panel. It contains:

Summary alert — total assigned, unassigned, and empty labs.

Action buttons:

Button	Action
Save report (CSV)	Downloads a CSV with every student's assignment, preference rank, tier, and advisor load
Save run (JSON)	Saves the full run (assignments, metrics, metadata) to the `results/` folder on the server for later analysis; shows a confirmation message inline
Export HTML	Saves a self-contained HTML snapshot of the current visualisation to `reports/`
Open Analysis	Navigates to the Analysis page

Summary badge row — always visible: `NPSS | PSI | Entropy | Load balance`. Load balance is the difference between the most-loaded and least-loaded advisor (including empty advisors), giving a quick measure of load spread.

Collapsible sections — both panels below are closed by default and can be expanded by clicking their header button:

- **[+] Advisor Popularity** — shows how many students listed each advisor as their 1st, 2nd, or 3rd choice, with tier breakdown.
- **[+] Metrics** — full metrics panel: NPSS, PSI, per-tier breakdown, advisor CPI entropy, and CPI skewness.

For `tiered_rounds`, the completion panel also includes a collapsible **Round-by-Round Trace** showing assignments, ties, and manual decisions per round.

2.5 Analysis page

The Analysis page (`/analysis`) is a dedicated post-allocation environment for comparing saved runs. It can be reached three ways:

- **Landing page** → **View Analysis** button
- **Completion panel** → **Open Analysis** button
- **Analysis page** → **Home** → re-select policy → **View Analysis**

Saving a run After finalising an allocation, click **Save run (JSON)** in the completion panel. The run is written to the `results/` folder as `{policy}_{timestamp}.json` and a confirmation message appears inline. No file dialog opens.

Loading runs on the Analysis page

1. **Run A** (required) — select a saved run from the dropdown.
2. **Run B** (optional) — select a second run for side-by-side comparison; leave as "None" for single-run view.
3. Click **Load**.

The page renders:

Section	What it shows
Metric scorecards	NPSS, PSI, per-tier breakdown, and advisor fairness metrics for each loaded run (side by side when two runs are loaded)
Per-Tier Mean Preference Rank chart	Grouped bar chart — one group per tier, one bar per policy. Y-axis is mean assigned preference rank (lower = better). Bar labels show % of tier assigned within their protected window.
Advisor Tier Heatmap	Capacity-normalised heatmap for each loaded run, showing how each advisor's load is distributed across CPI tiers

Navigation

- ← **Back to allocation** — returns to the allocation page with all state intact.
- **Home** — returns to the landing page for policy switching.

2.6 CLI usage (no GUI)

For scripted or batch runs, the allocation engine can be driven entirely from the command line. Note that `tiered_rounds` and `tiered_ll` require the `--auto-tiebreak` flag in CLI mode — without it the engine will refuse to run those policies and explain what to add.

Phase 0 only:

```
PYTHONPATH=src python -m allocator.allocation --phase0-only \
  --students data/sample_students.csv \
  --faculty  data/sample_faculty.csv \
  --out      reports/
```

Full allocation:

```
PYTHONPATH=src python -m allocator.allocation \
  --students data/sample_students.csv \
  --faculty  data/sample_faculty.csv \
  --policy   least_loaded \
  --out      reports/
```

The `--policy` flag accepts all five policies: `least_loaded`, `adaptive_ll`, `cpi_fill`, `tiered_rounds`, and `tiered_ll`. In CLI mode, tie-breaking in `tiered_rounds` and `tiered_ll` is resolved automatically by highest CPI (ties broken by student ID); the manual pick UI is only available in the Dash GUI.

---

3. Allocation Policies and Metrics

3.1 Shared foundation — Phase 0

All policies begin with **Phase 0**, which classifies students into tiers and sets advisor capacities. The output of Phase 0 is identical regardless of which policy follows.

Tiering Students are divided into tiers based on cohort CPI percentiles:

Tier	Percentile threshold	Preference window `N_tier` (S/F ≤ 4)	`N_tier` (S/F > 4)
A	$\geq$ 90th ($\pm$0.1 grace)	3	4
B	70th – 90th	5	6
C	< 70th	Full list	Full list

If more than 40% of students cluster in one band, tiering switches to **quartile mode** (A / B1 / B2 / C). If the cohort has fewer than 10 students, all are placed in Class A with `N_tier = 2`.

`N_tier` is used as the **assignment window** in `least_loaded`. For `cpi_fill` and `tiered_rounds` it is computed but not applied during assignment — it appears in the per-tier diagnostics only.

Faculty capacity Each advisor's `max_load` is set to `floor(S/F) + 1` unless a value is provided in the faculty CSV, where S is the number of students and F is the number of faculty.

3.2 Policy: Least Loaded (`least_loaded`)

Overview `least_loaded` is the default policy. It treats load balance as a co-equal constraint alongside preference satisfaction: at every assignment step, a student is directed to the **least-loaded eligible advisor** within their preference-protection window.

Pipeline

`Phase 0 → Round 1 → Main Allocation (Class A → B → C)`

Round 1 — Global first-choice pass Each advisor with at least one first-choice applicant selects exactly one student from that list, defaulting to the highest-CPI applicant (tie-broken by student ID). Manual overrides are supported. This initial pass seats top-tier students at popular advisors and establishes a baseline load distribution.

Main allocation Unassigned students are processed class by class:

1. **Class A** — each student is matched to the least-loaded advisor within their top `N_A` preferences that still has capacity. Students with no eligible advisor within `N_A` are promoted to Class B's pool.
2. **Class B** (original B plus any promoted Class A students) — same rule, window extends to `N_B`.
3. **Class C** (everyone remaining) — no window cap; all advisors with remaining capacity are eligible.

Assignment rule Given a student and their eligible advisors (within `N_tier`, with remaining capacity):

1. Find the **minimum current load** among eligible advisors.
2. If multiple advisors share that minimum, pick the one **earliest in the student's preference list**.
3. Assign the student and increment that advisor's load counter.

Load balance is the **primary criterion**; preference rank is the **tiebreaker**.

Properties

Property	Behaviour
Load balance	Strong — advisors are filled as evenly as possible at every step
Preference satisfaction (PSI)	Good — low-load advisors often coincide with early preferences
Merit sensitivity (NPSS)	Moderate — high-CPI students benefit from Round 1 and the tier window, but are not given explicit priority in the main allocation
Empty-lab guarantee	Indirect — the Class C full-list fallback keeps overflows near zero
Advisor CPI diversity (entropy)	High — load-spreading tends to distribute different CPI tiers across advisors
Robustness	High — performs consistently across random, clustered, and polarised cohorts

---

3.3 Policy: CPI Fill (`cpi_fill`)

Overview `cpi_fill` is a merit-first policy. It skips Round 1 and replaces the class-wise main allocation with a two-phase procedure built around a single principle: **students are processed**

in strict descending CPI order. Academically stronger students always gain access to their preferred advisors before lower-CPI peers.

Pipeline

`Phase 0` → `CPI-Fill Phase 1` → `CPI-Fill Phase 2`

Round 1 is not run. The tier information from Phase 0 is computed but not used during assignment — all students draw from their full preference list.

Phase 1 — CPI-ordered greedy assignment **Goal:** assign students in CPI order, stopping precisely when the number of remaining unassigned students equals the number of still-empty labs.

Stopping condition: `|unassigned| == |empty labs|`

Steps:

1. Sort all students by CPI, descending (ties broken by student ID).
2. For each student in that order:
   - Scan their preference list from rank 1 onward.
   - Assign the student to the **first advisor in their list with remaining capacity**.
   - Update loads. If `|unassigned| == |empty labs|`, stop Phase 1 immediately.
3. If all labs are already non-empty at the start, Phase 1 runs to completion with no stopping condition.
4. If `|unassigned| == |empty labs|` at the start, Phase 1 is skipped entirely.

Phase 2 — Empty-lab fill **Goal:** assign each remaining student to their most preferred advisor who currently has zero students.

Steps:

1. Sort remaining unassigned students by CPI, descending.
2. For each student, scan their preference list and assign to the **first advisor with `load == 0`**.

Properties

Property	Behaviour
Merit sensitivity (NPSS)	High — high-CPI students claim top choices before any lower-CPI student is considered
Preference satisfaction (PSI)	Mixed — top students get excellent matches; lower-CPI students may land further down their list
Load balance	Weaker — popular advisors can fill up quickly in Phase 1
Empty-lab guarantee	Very strong — structurally impossible to leave an advisor empty (when $S \geq F$)
Advisor CPI diversity (entropy)	Cohort-dependent

3.4 Policy: Adaptive Least Loaded (`adaptive_ll`)

Overview `adaptive_ll` is a variant of `least_loaded` that guarantees no empty labs when $S \geq F$, without requiring operator intervention. Before running the main allocation it adjusts the tier-window caps `N_A` and `N_B` iteratively until the post-A+B empty-lab count is at most the size of Class C, making full coverage structurally possible.

Pipeline

`Phase 0` → `Phase 0b (cap optimisation)` → `Round 1` → `Main Allocation (A → B → C)`

The assignment rule within each tier is identical to `least_loaded` (least-loaded advisor within window, preference rank as tiebreaker). The only difference is that `N_A` and `N_B` may be wider than the default values — expanded just enough to eliminate structural empty-lab risk.

Properties

Property	Behaviour
Load balance	Strong — same assignment rule as `least_loaded`
Preference satisfaction (NPSS/PSI)	Slightly lower than `least_loaded` when caps expand (students placed further along their list to reach under-subscribed advisors)
Empty-lab guarantee	Yes — structural (when $S \geq F$)
Operator involvement	None required
Robustness	Reliable across sparse cohorts ($S/F \approx 1$) where `least_loaded` may leave empty labs

> **When to prefer `adaptive_ll` over `least_loaded`:** when the S/F ratio is close to 1 and empty labs are unacceptable. At $S/F > 1.3$ the two policies are usually identical (no cap expansion needed).

3.5 Policy: Tiered LL (`tiered_ll`)

Overview `tiered_ll` is a hybrid policy that combines the transparency of `tiered_rounds` with the coverage guarantee of `adaptive_ll`. It runs interactive tiered rounds for the first k rounds (determined by a dry-run pre-computation), then switches automatically to a two-phase backfill for any remaining students.

Pipeline

```
Phase 0a (tiering)  →  Phase 0b (dry-run → k_crit)
  →  Phase 1: Tiered Rounds 1..k  (interactive)
  →  Phase 2a: Excess assignment  (automatic)
  →  Phase 2b: Empty-lab fill  (automatic)
```

Finding the critical round k Before any interactive steps a full dry-run of `tiered_rounds` is run with automatic CPI tie-breaking. The dry-run scans each round's outcome and stops at the first

round n that meets a criterion:

Condition after round n	k returned	Reason
Unreachable faculty or stall	max(prev round, 1)	Structural problem; don't run the broken round
`unassigned < empty_labs`	max(prev round, 0)	Overshoot — round n leaves more empty labs than students; stop one round earlier
`unassigned == empty_labs`	round n	Exact parity — this is the right stopping point

$k = 0$ is valid: round 1 itself overshoots, so no tiered rounds are run and the full preference list is used for backfill.

Phase 2 — Two-phase backfill (GUI) **Phase 2a** (while total_unassigned > empty_labs): students are processed in descending CPI order. Each student is assigned to their highest-preferred advisor in `prefs[k:]` that (a) has remaining capacity and (b) has not already received a student in Phase 2a (one-per-advisor discipline, mirroring Phase 1). The stop condition is re-evaluated after every individual assignment; overshoot is structurally impossible. Students with no eligible advisor are deferred to Phase 2b.

Phase 2b (when total_unassigned $\leq$ empty_labs): the combined queue of remaining + deferred students is re-sorted by descending CPI, then each student is assigned to their highest-preferred empty lab in `prefs[k:]`. Once a lab is filled it leaves the candidate set. Students whose `prefs[k:]` contains no empty lab become overflow.

In CLI auto-mode, Phase 2a uses `cpi_fill_phase1` on `prefs[k:]` (no one-per-advisor constraint) and Phase 2b uses `cpi_fill_phase2` on the full preference list.

Properties

Property	Behaviour
Transparency	High — rounds 1..k are fully logged and interactive
Preference satisfaction (NPSS)	Depends on k; low k (e.g. k=1) collapses toward `cpi_fill` behaviour
Empty-lab guarantee	Yes — Phase 2b fills all empty labs when S $\geq$ F
Operator involvement	Required for CPI ties in rounds 1..k; Phase 2 is automatic

Property	Behaviour
Load balance	Phase 1 has one-per-advisor-per-round discipline; Phase 2a has one-per-advisor-per-phase discipline

---

3.6 Policy: CPI-Tiered Preference Rounds (`tiered_rounds`)

Overview `tiered_rounds` is a **round-based, interactive policy**. In round *n*, every unassigned student simultaneously offers their *n*-th preference. Each participating advisor selects at most one student (the highest-CPI candidate). If two or more students share the top CPI at the same advisor, the operator resolves the tie manually. The process repeats until all students are assigned.

Pipeline

```
Phase 0  →  Preference Rounds (n = 1, 2, 3, …)
```

No separate Round 1 or class-wise main allocation. Phase 0 tier classification is computed but does not constrain round participation.

Selection rule

- **Unique highest CPI** → immediate assignment.
- **CPI tie at the top** → operator picks one; remaining candidates advance to round *n+1*.
- Advisors that became full in a previous round are skipped entirely.
- Each advisor allocates **at most one student per round**, even if it still has capacity > 1.

Properties

Property	Behaviour
Merit sensitivity (NPSS)	High — CPI determines priority within each round
Preference satisfaction (PSI)	High in balanced cohorts; students advance down their list only when earlier choices are taken
Transparency	Highest — every assignment and every tie-break decision is logged in the round trace
Operator involvement	Required for CPI ties; cannot run unattended if ties exist
Load balance	Implicit (one per advisor per round); not explicitly optimised

---

3.7 Policy comparison

Aspect	`least_loaded`	`adaptive_ll`	`cpi_fill`	`tiered_rounds`	`tiered_ll`
Round 1	Yes	Yes	No	No	No
Processing order	Tier-by-tier	Tier-by-tier	Strict descending CPI	Round-by-preference-rank	Rounds 1..k, then backfill
Preference window	N_tier per tier	N_tier (auto-widened)	Full list	Full list (N_tier diagnostic)	Full list
Primary assignment criterion	Min load	Min load	First pref with capacity	Highest CPI in round	CPI in rounds; Phase 2a: first pref with capacity (one per advisor); Phase 2b: first empty lab
Tie-breaking	Preference rank	Preference rank	Student ID	Highest CPI (CLI) / Manual pick (GUI)	Highest CPI (CLI) / Manual pick in GUI rounds
Empty-lab guarantee	Indirect	Yes (when $S \geq F$)	Explicit (Phase 2)	No	Yes (when $S \geq F$)
Merit sensitivity (NPSS)	Moderate	Moderate	High	High	Depends on k; low k $\rightarrow$ similar to cpi_fill
Equal-weighted satisfaction (PSI)	Cohort-dependent	Cohort-dependent	Cohort-dependent	High in balanced cohorts	Cohort-dependent
Load balance	Strong	Strong	Variable	Implicit (one per advisor per round)	Phase 1 + Phase 2a: one per advisor per phase; Phase 2b: structured
CLI available	Yes	Yes	Yes	Yes (auto CPI ties)	Yes (auto CPI ties)

Note: PSI and advisor entropy outcomes are cohort-sensitive. Neither policy dominates consistently across all cohort types. See Section 3.8 for guidance on interpretation.

3.8 Metrics

The app reports metrics in two places: the **Statistics** tab of the replay panel (updated at each step) and the **completion panel summary badge row** (final state only).

NPSS — Normalized Preference Satisfaction Score NPSS is the **primary metric**. It measures how well student preferences were honoured, weighted by CPI.

Per-student score:

$$\text{score}_i = \frac{F - p_i + 1}{F}$$

where p_i is the preference rank of the advisor assigned to student i (1 = 1st choice) and F is the total number of faculty (= the full preference list length, the same for all students).

Aggregate:

$$\text{NPSS} = \sum_{i=1}^{S} \frac{\text{CPI}_i}{\sum_j \text{CPI}_j} \cdot \text{score}_i$$

NPSS lies in (0, 1]. Using F as the denominator — rather than the tier-specific `N_tier` — means all policies are evaluated on the same scale, enabling fair cross-protocol comparison. A student assigned at rank 1 always scores 1.0; a student at rank F scores $1/F \approx 0$.

NPSS range	Interpretation
0.95 – 1.00	Excellent — cohort is landing predominantly at ranks 1–2
0.85 – 0.94	Good — most students in top 5; some further
0.70 – 0.84	Moderate — noticeable preference compromise; check per-tier breakdown
< 0.70	Poor — systematic mismatch; review capacity parameters or preference diversity

The CPI weighting reflects the protocol's philosophy that failures to honour high-CPI students' preferences should penalise the score more than equivalent failures for lower-CPI students.

PSI — Preference Satisfaction Index PSI is a **complementary, equal-weighted metric**. It treats every student identically and asks: on average, how close to the top of their list did each student land?

Per-student score:

$$\text{PSI}_i = 1 - \frac{p_i - 1}{F - 1}$$

This maps 1st choice $\rightarrow$ 1.0 and last choice $\rightarrow$ 0.0, normalised by $F - 1$ so comparisons are valid across cohorts of different sizes.

Aggregate:

$$\text{mean PSI} = \frac{1}{S} \sum_{i=1}^{S} \text{PSI}_i$$

PSI is particularly useful when comparing policies: because `cpi_fill` processes students in CPI order, it can boost NPSS for high-CPI students while simultaneously lowering mean PSI for lower-CPI students. PSI makes this redistribution visible.

Mean PSI range	Interpretation
0.90 – 1.00	Excellent — students landing very near their 1st or 2nd choice on average
0.75 – 0.89	Good — most students in their top quarter of preferences
0.60 – 0.74	Moderate — noticeable preference compromise across the cohort
< 0.60	Poor — students landing in the lower half of their list on average

Reading NPSS and PSI together:

Pattern	Meaning
NPSS high, PSI lower than expected	Top-CPI students well served; lower-CPI students landing further down their lists
PSI high, NPSS lower	Cohort lands near top preferences on average, but CPI-weighted satisfaction is lower
Both high	Allocation working well across all dimensions
Both low	Systematic preference mismatch; investigate capacity and preference diversity

Overflow Count (diagnostic) The overflow count reports the number of students assigned **beyond their tier `N_tier` window** (rank > `N_tier`). This is a protocol-compliance diagnostic, tracked separately from NPSS. Because NPSS now uses F as its denominator, out-of-window placements still receive a positive NPSS contribution — the overflow count is the only place where tier-cap compliance is flagged numerically.

All policies can produce overflow, but through different mechanisms:

- **`least_loaded`**: the N_tier window is applied first; a student is only promoted to a wider pool (B → C, or A → B → C) when every advisor within their current window is at capacity. Overflow occurs when the promotion chain ends at the Class C full-list round and the assigned rank exceeds the original N_tier. This is anomalous — it only happens under very tight capacity — and is flagged as a **red warning badge**.
- **`cpi_fill` / `tiered_rounds`**: no N_tier window is applied during assignment; overflow is a structural feature of the protocol. For `tiered_rounds` in particular, a student reaches round n naturally when their first $n-1$ preferences were claimed by others, so assignments beyond N_tier are expected and routine. These are shown as a **blue informational badge**, not an error.

Advisor Satisfaction Metrics **MSES — Mean Student Enthusiasm Score** is the primary advisor satisfaction metric. For each advisor, it is the mean preference rank at which their assigned students listed them:

$$\text{MSES}(a) = \frac{1}{n_a} \sum_{i \in \mathcal{S}_a} \text{rank}(a, i)$$

The system-level score is the mean MSES across all assigned advisors. Lower is better — a value near 1.0 means students were predominantly assigned to advisors they ranked 1st.

Avg MSES	Interpretation
≤ 2.0	Students are highly enthusiastic about their advisor assignments
≤ 4.0	Good match overall
> 4.0	Students landing further down their lists; check capacity or preference diversity

Avg Load Utilization (LUR) is the mean of `actual_load / max_load` across assigned advisors, reported as a percentage. It is a **capacity utilisation signal** — it summarises how fully the allocated seats are being used — not an equity measure. Two allocations with very different load spreads can have identical Avg LUR. Per-advisor LUR appears in the Tier Heatmap y-axis labels.

Advisor Equity — Load Distribution These metrics measure whether students were spread fairly across advisors.

Empty Labs is the count of advisors who received no students. Shown as a badge in the completion panel and metrics section.

Load Balance = `max(advisor loads) − min(advisor loads)` across **all** advisors including empty labs. A value of 0 means identical loads; 1 is the minimum non-zero spread and is typical in well-balanced runs.

Advisor Equity — Tier Mixing These metrics measure whether each advisor received a diverse cross-section of CPI tiers.

Per-advisor entropy $H_{\text{norm}}(a)$ is the normalized Shannon entropy of the CPI-tier distribution within advisor *a*'s cohort:

$$H_{\text{norm}}(a) = \frac{-\sum_{k=1}^{K} p_k(a)\, \log p_k(a)}{\log K}$$

where K is the number of tiers (3 in percentile mode, 4 in quartile mode) and $p_k(a)$ is the fraction of *a*'s students in tier *k*. $H_{\text{norm}} \in [0, 1]$: 0 when all students are in the same tier, 1 when perfectly uniform across all tiers.

The system-level figure $\overline{H}_{\text{norm}}$ sums over all *F* faculty (empty labs contribute 0) and divides by *F*.

Load-Aware Entropy Ceiling H_{baseline} is the tightest upper bound on $\overline{H}_{\text{norm}}$ for this run's actual load distribution. The per-advisor ceiling is:

$$H_{\text{max}}(n) = \frac{\log \min(n,\ K)}{\log K}$$

since an advisor with n students can represent at most $\min(n, K)$ distinct tiers. The system-level ceiling averages this over all *F* advisors using their actual loads (empty labs contribute 0):

$$H_{\text{baseline}} = \frac{1}{F} \sum_{a=1}^{F} H_{\text{max}}(\text{actual\_load}(a))$$

This guarantees $\overline{H}_{\text{norm}} \leq H_{\text{baseline}}$, so ERR is always in [0 %, 100 %].

Equity Retention Rate (ERR) measures what fraction of that ceiling the policy actually achieved:

$$\text{ERR} = \frac{\overline{H}_{\text{norm}}}{H_{\text{baseline}}} \times 100\,\%$$

ERR answers "given how this run distributed students across advisors, what fraction of the maximum possible tier mixing was achieved?" Because the baseline reflects the actual load distribution, read ERR together with Empty Labs and Load Balance for the full equity picture.

ERR	Interpretation
$\geq$ 80%	Policy used most of its tier-mixing headroom
60–80%	Moderate mixing; some tier concentration within advisors
< 60%	Significant tier concentration introduced by the policy

CPI Skewness *(diagnostic)* measures asymmetry in the distribution of advisor mean CPIs. Fisher-Pearson formula, std-normalized and scale-invariant. $|\gamma|$ `< 0.5` is acceptable; $|\gamma|$ `> 1.0` warrants investigation. Use alongside ERR, not as a standalone equity verdict.

3.9 How to read the metrics together

When interpreting a single run or comparing policies, use this hierarchy:

1. **NPSS** *(primary student metric)* — start here. It directly measures whether the allocation honours student preferences, weighted by CPI. All policies use the same full-list denominator, so NPSS values are directly comparable across policies.
2. **PSI** *(secondary student metric)* — check for redistribution effects. If NPSS and PSI move in opposite directions across policies, the allocation is trading higher CPI-weighted satisfaction for lower equal-weighted satisfaction (or vice versa). This is a value judgement, not a failure.
3. **MSES** *(primary advisor satisfaction metric)* — check whether students were matched to advisors they genuinely sought out. A rising MSES across policies indicates students are landing further from their preferences on the advisor's side.
4. **LUR** *(capacity utilisation)* — how fully are allocated seats being used? Not an equity measure; two runs with very different load spreads can have identical Avg LUR.
5. **Empty Labs + Load Balance** *(load distribution equity)* — check whether students were spread fairly. Empty Labs flags advisors bypassed entirely; Load Balance gives the full spread. These two together capture load equity independently of tier composition.
6. **Equity Retention Rate + Entropy Ceiling** *(tier mixing equity)* — given how students were distributed, how much of the possible tier diversity was achieved? Read ERR alongside the ceiling: a high ERR under a low ceiling (few students per advisor) is different from a high ERR under a high ceiling.
7. **Advisor Tier Distribution Heatmap** *(visual diagnostic)* — shows which tiers each advisor's cohort contains, row-normalised by capacity. Use it to understand *why* ERR is high or low for specific advisors.
8. **CPI Skewness** *(diagnostic)* — cross-check for CPI concentration in the advisor mean-CPI distribution. Already std-normalized (scale-invariant). $|\gamma|$ `> 1.0` warrants attention. Use alongside ERR, not as a standalone verdict.
9. **Overflow Count** *(protocol-compliance diagnostic)* — reports placements beyond the tier `N_tier` window. For window-applying policies (`least_loaded`) this signals near-overflow stress. For `cpi_fill` and `tiered_rounds` — which apply no window during assignment — it is purely informational (blue badge, not red). It does not affect NPSS.

If metrics disagree: this is normal. The policies trade off across dimensions rather than one uniformly dominating. The choice is a value judgement about institutional priorities — merit-weighted access, equitable treatment, load balance, or operator transparency — not a metric-determined optimum.

For further details see `MSThesisAllocationProtocol.md` (full protocol specification), `NPSS_Metric.md` (NPSS/PSI definitions), `docs/policy_*.md` (per-policy deep-dives), and `stats/policy_report.md` / `stats/policy_report.pdf` (empirical comparison across all five policies on two real cohorts and four synthetic datasets).